

**Documentación Técnica y Diseño de Pruebas del Sistema SmartComedor**

Sandro Jaramillo y Anuar Sierra y Gabriel Macias

Universidad Popular del Cesar

Facultad de Ingenierías y Tecnologías

Ingeniería de Software II

Docente: Maribel Romero

Valledupar, 2026

## Documentación Técnica y Diseño de Pruebas del Sistema SmartComedor

Este documento presenta la documentación técnica integral del sistema **SmartComedor**, una aplicación web de tres capas para la gestión del comedor universitario (registro y validación de elegibilidad de estudiantes, gestión de pagos, control de asistencia, retroalimentación del servicio y auditoría). La primera parte describe el problema, el sistema, el modelo de requerimientos (funcionales y no funcionales), el modelo de casos de uso y el modelo de diseño (clases, secuencias, entidad-relación y componentes). La segunda parte documenta el proceso de verificación y validación: planificación, pruebas unitarias (caja negra y caja blanca), pruebas de integración (incremental y basada en hilos), pruebas de sistema (seguridad, rendimiento, usabilidad y portabilidad) y pruebas de aceptación. La suite automatizada ejecuta **196 pruebas con 100 % de éxito** y una cobertura global aproximada del **80,4 %** de sentencias. El trabajo se apoya en estándares y técnicas reconocidas de la ingeniería de software (IEEE, 2008; International Organization for Standardization, 2011; Myers y col., 2011; Pressman & Maxim, 2014).

*Palabras clave:* pruebas de software, verificación y validación, casos de uso, requerimientos, calidad de software, JWT, SISBÉN.

## Índice

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>PRIMERA PARTE. Descripción del Sistema</b>            | <b>6</b> |
| <b>Descripción del sistema</b>                           | <b>6</b> |
| Identificación del problema . . . . .                    | 6        |
| Contexto . . . . .                                       | 6        |
| Problemas concretos detectados . . . . .                 | 6        |
| Planteamiento del problema . . . . .                     | 7        |
| Justificación . . . . .                                  | 7        |
| Descripción detallada del sistema o aplicación . . . . . | 8        |
| Propósito y alcance . . . . .                            | 8        |
| Actores y roles . . . . .                                | 8        |
| Arquitectura general . . . . .                           | 8        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| DOCUMENTACIÓN TÉCNICA — SMARTCOMEDOR                       | 3         |
| Stack tecnológico . . . . .                                | 9         |
| Reglas de negocio principales . . . . .                    | 10        |
| <b>Modelo de requerimientos</b>                            | <b>10</b> |
| Descripción . . . . .                                      | 10        |
| Requisitos funcionales . . . . .                           | 10        |
| Requisitos no funcionales . . . . .                        | 14        |
| <b>Modelo de casos de uso</b>                              | <b>16</b> |
| Diagramas de caso de uso . . . . .                         | 16        |
| Descripción de casos de uso . . . . .                      | 18        |
| CU-01 — Registrar estudiante . . . . .                     | 19        |
| CU-02 — Iniciar sesión . . . . .                           | 19        |
| CU-07 — Validar SISBÉN . . . . .                           | 20        |
| CU-11 — Subir pago y comprobantes . . . . .                | 20        |
| CU-14 — Registrar almuerzo (caso de uso central) . . . . . | 20        |
| Otros casos de uso . . . . .                               | 21        |
| <b>Modelo de diseño del sistema</b>                        | <b>21</b> |
| Diagrama de clases detallado . . . . .                     | 21        |
| Diagramas de secuencias . . . . .                          | 24        |
| Diagrama entidad–relación . . . . .                        | 29        |
| Diagrama de componentes . . . . .                          | 31        |
| <b>Producto del software</b>                               | <b>32</b> |
| Despliegue . . . . .                                       | 32        |
| Estructura del producto . . . . .                          | 33        |
| Características destacadas . . . . .                       | 33        |
| <b>SEGUNDA PARTE. Pruebas del Software</b>                 | <b>33</b> |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Introducción</b>                                       | <b>33</b> |
| <b>Planificación de las pruebas</b>                       | <b>34</b> |
| Objetivos de las pruebas . . . . .                        | 34        |
| Alcance de las pruebas . . . . .                          | 34        |
| Estrategias de pruebas . . . . .                          | 35        |
| Ambiente de pruebas . . . . .                             | 36        |
| <b>Pruebas unitarias</b>                                  | <b>36</b> |
| Análisis de las pruebas . . . . .                         | 36        |
| Diseño de casos de prueba . . . . .                       | 37        |
| Caja negra — Clases de equivalencia (Login) . . . . .     | 37        |
| Caja blanca — Cobertura de caminos (login) . . . . .      | 38        |
| Caja negra — Valores límite (registro y montos) . . . . . | 39        |
| Registro de almuerzo y middleware . . . . .               | 39        |
| Ejecución y evaluación de las pruebas . . . . .           | 40        |
| Resultado de ejecución . . . . .                          | 40        |
| Cobertura de código . . . . .                             | 40        |
| Evaluación . . . . .                                      | 41        |
| <b>Pruebas de integración</b>                             | <b>42</b> |
| Estrategia de pruebas incrementales . . . . .             | 42        |
| Descripción . . . . .                                     | 42        |
| Diseño y ejecución de casos . . . . .                     | 42        |
| Estrategia de pruebas basadas en hilos . . . . .          | 43        |
| Descripción . . . . .                                     | 43        |
| Diseño y ejecución de casos . . . . .                     | 43        |
| <b>Pruebas de sistema</b>                                 | <b>44</b> |
| Pruebas de seguridad . . . . .                            | 44        |
| Diseño . . . . .                                          | 44        |

|                                               |           |
|-----------------------------------------------|-----------|
| Ejecución y evaluación . . . . .              | 45        |
| Pruebas de rendimiento . . . . .              | 46        |
| Diseño . . . . .                              | 46        |
| Ejecución y evaluación . . . . .              | 46        |
| Pruebas de usabilidad . . . . .               | 46        |
| Diseño . . . . .                              | 46        |
| Ejecución y evaluación . . . . .              | 47        |
| Pruebas de portabilidad . . . . .             | 47        |
| Diseño . . . . .                              | 47        |
| Ejecución y evaluación . . . . .              | 48        |
| <b>Pruebas de aceptación</b>                  | <b>48</b> |
| Diseño de casos de prueba . . . . .           | 48        |
| Ejecución y evaluación de la prueba . . . . . | 49        |
| <b>Conclusiones</b>                           | <b>50</b> |

# PRIMERA PARTE. Descripción del Sistema

## Descripción del sistema

### Identificación del problema

#### *Contexto*

El programa de bienestar universitario ofrece un servicio de comedor (almuerzos subsidiados) dirigido a estudiantes en condición de vulnerabilidad socioeconómica. La elegibilidad para el subsidio depende de la clasificación del estudiante en el SISBÉN (Sistema de Identificación de Potenciales Beneficiarios de Programas Sociales del Estado colombiano), de su situación académica (carrera, semestre) y de su situación personal. Históricamente la operación se gestionaba mediante procesos manuales y en papel: el estudiante diligenciaba formularios físicos y entregaba fotocopias; un funcionario validaba manualmente que la cédula y los nombres del certificado SISBÉN correspondieran al documento de identidad; el control de asistencia se llevaba en planillas impresas, y el pago/recarga de almuerzos se soportaba con comprobantes físicos. No existía trazabilidad ni un canal estructurado de retroalimentación.

#### *Problemas concretos detectados*

La Tabla 1 resume los problemas identificados y sus consecuencias.

**Tabla 1***Problemas identificados en la operación manual del comedor*

| #  | Problema                                              | Consecuencia                                         |
|----|-------------------------------------------------------|------------------------------------------------------|
| P1 | Validación manual de elegibilidad (SISBÉN vs. cédula) | Lentitud, error humano y riesgo de fraude documental |
| P2 | Control de asistencia en papel                        | Doble cobro, suplantación y pérdida de información   |
| P3 | Conciliación manual de pagos/recargas                 | Descuadres contables y comprobantes extra-viados     |
| P4 | Ausencia de trazabilidad/auditoría                    | Imposibilidad de investigar incidentes               |
| P5 | Falta de canal de retroalimentación                   | Desconocimiento de la percepción del servicio        |
| P6 | Gestión de cupos y ciclos sin automatizar             | Beneficios vencidos que siguen activos               |
| P7 | Información dispersa entre actores                    | Ausencia de una fuente única de verdad               |

***Planteamiento del problema***

¿Cómo digitalizar y automatizar la gestión integral del comedor universitario —registro y validación de elegibilidad, control de asistencia, gestión de pagos, retroalimentación del servicio y auditoría— de manera segura, trazable y verificable, reduciendo el trabajo manual y el riesgo de fraude?

***Justificación***

Un sistema de información web que centralice estos procesos reduce el fraude (validación automática del SISBÉN por OCR/parseo de PDF e identificación con código QR único por estudiante), garantiza trazabilidad (bitácoras `audit_logs` y `supervisor_logs`), acelera la operación (importación masiva desde Excel, cálculo automático de almuerzos y cierre automático de ciclos) y mejora la calidad del servicio (calificaciones y quejas/sugerencias/comentarios).

## Descripción detallada del sistema o aplicación

### *Propósito y alcance*

**SmartComedor** permite registrar estudiantes, gestionar pagos, supervisar almuerzos y administrar el servicio de comedor universitario, con control de acceso por roles, validación automática de elegibilidad y auditoría completa. El alcance funcional abarca nueve módulos: autenticación y cuentas; gestión de estudiantes; gestión de supervisores; pagos/recargas; asistencia (almuerzos); ciclos; retroalimentación; noticias; y auditoría.

### *Actores y roles*

El sistema define cuatro roles (enum `UserRole`), descritos en la Tabla 2.

**Tabla 2**

*Actores y roles del sistema*

| Rol             | Valor                         | Descripción                                                                                                         |
|-----------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Administrador   | <code>admin</code>            | Configura el sistema; gestiona estudiantes, supervisores, pagos, ciclos y noticias; responde quejas; valida SISBÉN. |
| Supervisor      | <code>supervisor</code>       | Registra la asistencia/almuerzo de los estudiantes; busca estudiantes y consulta su historial.                      |
| Estudiante      | <code>student</code>          | Se registra; consulta perfil y almuerzos disponibles; sube pagos; califica el servicio; envía quejas; lee noticias. |
| Auditor externo | <code>external_auditor</code> | Acceso de solo lectura a auditoría, supervisores, pagos, asistencia y analítica.                                    |

### *Arquitectura general*

El sistema sigue una arquitectura cliente-servidor de tres capas desplegada con Docker Compose (Figura 1). El frontend (React + Vite + Material UI) consume una API REST (Express + TypeScript) que persiste en PostgreSQL mediante el ORM Sequelize. El backend integra servicios de correo (Nodemailer), validación SISBÉN (pdf-parse y OCR con tesseract.js),



[illegible]

*Arquitectura general del sistema SmartComedor.*

La Tabla 3 resume las tecnologías por capa.

## Stack tecnológico

| Capa / aspecto         | Tecnología                                                                     |
|------------------------|--------------------------------------------------------------------------------|
| Frontend               | React 18 + TypeScript + Vite + Material UI (MUI)                               |
| Backend                | Node.js + Express + TypeScript (API REST bajo <code>/api</code> )              |
| Base de datos          | PostgreSQL 14 con ORM Sequelize                                                |
| Autenticación          | JWT (access + refresh); contraseñas con bcrypt                                 |
| Segundo factor         | TOTP (speakeasy + qrcode)                                                      |
| Correo                 | Nodemailer (SMTP)                                                              |
| OCR / PDF              | tesseract.js, pdf-parse                                                        |
| Archivos / importación | multer; xlsx                                                                   |
| Programación de tareas | node-cron                                                                      |
| Contenedores           | Docker Compose (perfiles <code>dev</code> y <code>prod</code> , nginx en prod) |

### *Reglas de negocio principales*

- **Precio del almuerzo:** `MEAL_PRICE` = 2000 (COP). Los almuerzos incluidos en un pago se calculan como  $\lfloor \text{monto} / 2000 \rfloor$ ; el residuo ( $\text{monto} \bmod 2000$ ) no genera almuerzo.
- **Monto mínimo de recarga:** un pago debe ser  $\geq 2000$ ; en caso contrario se rechaza con HTTP 400.
- **Almuerzos disponibles:** se calculan sobre los pagos *verificados* como  $\sum(\text{mealsIncluded} - \text{mealsUsed})$ .
- **Un almuerzo por día:** un estudiante no puede registrar dos almuerzos el mismo día.
- **Días autorizados:** el almuerzo solo se registra en los días del comedor asignados al estudiante (`diasComedor`).
- **Elegibilidad SISBÉN:** las categorías válidas pertenecen a los grupos A, B y C; la validación coteja cédula, nombre y apellido contra la cédula frontal.
- **Autorización de cuentas:** un usuario con `isAuthorized` = `false` no puede iniciar sesión.
- **Invitación de supervisores:** el enlace de invitación es válido por 48 horas.
- **Ciclos:** un ciclo vencido (`endDate` < hoy) con estado **Activo** se cierra automáticamente.

## Modelo de requerimientos

### Descripción

El modelo de requerimientos especifica qué debe hacer el sistema (requisitos funcionales, RF) y bajo qué condiciones de calidad (requisitos no funcionales, RNF) (Sommerville, 2011). Los requisitos se derivaron del análisis del problema, de las rutas REST implementadas, de los controladores y de las reglas de negocio verificadas por las pruebas. Se emplea la convención RF-XX y RNF-XX, con prioridad Alta, Media o Baja.

### Requisitos funcionales

**Tabla 4***Requisitos funcionales del sistema*

| <b>ID</b> | <b>Requisito</b>                                                                                                                                                       | <b>Prioridad</b> |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| RF-01     | Registro de estudiante con datos personales/académicos y carga de archivos (SISBÉN, cédula frontal, horario PDF, recibo de pago).                                      | Alta             |
| RF-02     | Iniciar sesión con email, contraseña y rol, retornando access token y refresh token JWT.                                                                               | Alta             |
| RF-03     | Rechazar el inicio de sesión ante campos faltantes (400), credenciales inválidas (401), usuario inactivo (401), rol no coincidente (403) o cuenta no autorizada (403). | Alta             |
| RF-04     | Refrescar el access token a partir de un refresh token válido.                                                                                                         | Alta             |
| RF-05     | Consultar y actualizar el perfil del usuario autenticado.                                                                                                              | Media            |
| RF-06     | Cambiar la contraseña validando la contraseña actual.                                                                                                                  | Media            |
| RF-07     | Recuperar la contraseña por correo con token, sin revelar si el email existe.                                                                                          | Alta             |
| RF-08     | Restablecer la contraseña con un token válido y no expirado.                                                                                                           | Alta             |
| RF-09     | Configurar y verificar 2FA (TOTP) exclusivamente para estudiantes.                                                                                                     | Media            |
| RF-10     | Listar estudiantes con paginación y búsqueda.                                                                                                                          | Alta             |
| RF-11     | Consultar un estudiante por ID, incluyendo sus almuerzos disponibles.                                                                                                  | Alta             |
| RF-12     | Crear un estudiante y su usuario asociado.                                                                                                                             | Alta             |
| RF-13     | Actualizar y deshabilitar (soft delete) un estudiante.                                                                                                                 | Alta             |
| RF-14     | Importar estudiantes masivamente desde Excel (.xlsx).                                                                                                                  | Alta             |

*continúa en la siguiente página*

Tabla 4 – *continuación*

| ID    | Requisito                                                                          | Prioridad |
|-------|------------------------------------------------------------------------------------|-----------|
| RF-15 | Buscar estudiantes (por cédula y otros filtros) para el registro de almuerzo.      | Alta      |
| RF-16 | Calcular los almuerzos disponibles a partir de los pagos verificados.              | Alta      |
| RF-17 | Validar el SISBÉN cotejando cédula, nombre y apellido contra la cédula frontal.    | Alta      |
| RF-18 | Actualizar el ciclo de un estudiante (revalidación/deshabilitación).               | Media     |
| RF-19 | Listar supervisores y sus bitácoras.                                               | Media     |
| RF-20 | Crear un supervisor con contraseña temporal.                                       | Alta      |
| RF-21 | Actualizar, deshabilitar y cambiar el estado (activo/autorizado) de un supervisor. | Media     |
| RF-22 | Generar un enlace de invitación válido por 48 h para auto-registro de supervisor.  | Media     |
| RF-23 | Unirse mediante invitación validando el token.                                     | Media     |
| RF-24 | Asignar y remover estudiantes a un supervisor, evitando duplicados.                | Media     |
| RF-25 | Consultar los estudiantes asignados a un supervisor.                               | Media     |
| RF-26 | Crear un pago asociado a un estudiante, calculando los almuerzos incluidos.        | Alta      |
| RF-27 | Cargar los comprobantes del pago (comprobante, recibo universidad, recibo banco).  | Alta      |
| RF-28 | Calcular los almuerzos de un monto, rechazando montos < \$2000.                    | Alta      |

*continúa en la siguiente página*

Tabla 4 – *continuación*

| ID    | Requisito                                                                                                          | Prioridad |
|-------|--------------------------------------------------------------------------------------------------------------------|-----------|
| RF-29 | Listar pagos con paginación y filtros (por estudiante y por estado).                                               | Alta      |
| RF-30 | Verificar un pago (marcar <code>isVerified</code> , registrar <code>verifiedBy</code> y <code>verifiedAt</code> ). | Alta      |
| RF-31 | Registrar el almuerzo validando estudiante activo, día autorizado, no haber comido hoy y disponibilidad.           | Alta      |
| RF-32 | Decrementar la disponibilidad y registrar un <code>SupervisorLog</code> al registrar el almuerzo.                  | Alta      |
| RF-33 | Consultar el historial de asistencia con filtros por estudiante y fechas.                                          | Media     |
| RF-34 | Consultar la asistencia del día.                                                                                   | Media     |
| RF-35 | Ofrecer analítica del tablero (métricas agregadas).                                                                | Media     |
| RF-36 | Crear, listar y consultar ciclos.                                                                                  | Media     |
| RF-37 | Cerrar un ciclo manualmente.                                                                                       | Media     |
| RF-38 | Cerrar automáticamente los ciclos vencidos y gestionar la revalidación.                                            | Media     |
| RF-39 | Calificar el servicio (1–5 estrellas + comentario opcional).                                                       | Media     |
| RF-40 | Consultar calificaciones y su promedio/distribución.                                                               | Baja      |
| RF-41 | Crear quejas, sugerencias o comentarios, de forma anónima o identificada.                                          | Media     |
| RF-42 | Responder y marcar como resueltas las quejas.                                                                      | Media     |
| RF-43 | Publicar, editar y eliminar (soft delete) noticias, y listarlas.                                                   | Baja      |
| RF-44 | Registrar en auditoría las acciones y eventos HTTP sensibles.                                                      | Alta      |

*continúa en la siguiente página*

Tabla 4 – *continuación*

| ID    | Requisito                                                                                    | Prioridad |
|-------|----------------------------------------------------------------------------------------------|-----------|
| RF-45 | Consultar y filtrar la auditoría (acción, método, código, ruta, rol, fechas) con paginación. | Alta      |

### Requisitos no funcionales

Los RNF se organizan según el modelo de calidad ISO/IEC 25010 (International Organization for Standardization, [2011](#)).

Tabla 5

*Requisitos no funcionales del sistema*

| ID     | Categoría   | Requisito                                                                         |
|--------|-------------|-----------------------------------------------------------------------------------|
| RNF-01 | Seguridad   | Contraseñas almacenadas cifradas con bcrypt (hash + salt).                        |
| RNF-02 | Seguridad   | Control de acceso por JWT (access 15 min, refresh 7 días) y autorización por rol. |
| RNF-03 | Seguridad   | Rechazo de peticiones sin token (401) y de rol no permitido (403).                |
| RNF-04 | Seguridad   | No filtrar trazas de error (stack traces); mensajes genéricos.                    |
| RNF-05 | Seguridad   | Recuperación de contraseña sin revelar si un correo existe.                       |
| RNF-06 | Seguridad   | Cabeceras de seguridad (nosniff, anti-clickjacking) y sin X-Powered-By.           |
| RNF-07 | Seguridad   | Validación de filtros de auditoría contra listas blancas.                         |
| RNF-08 | Seguridad   | 2FA (TOTP) como segundo factor para estudiantes.                                  |
| RNF-09 | Rendimiento | Consultas críticas y autenticación responden en menos de 3 s.                     |

Tabla 5 – *continuación*

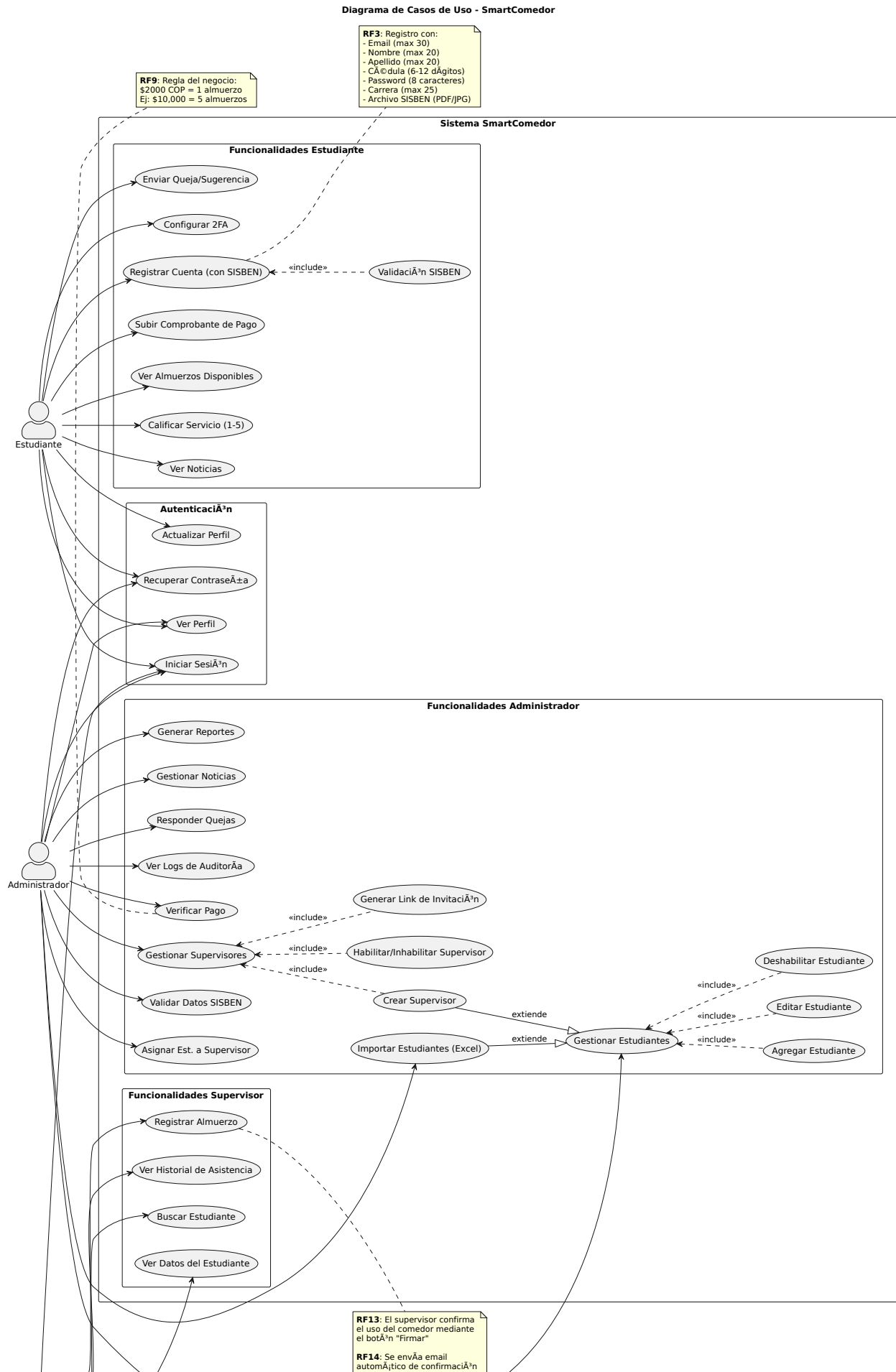
| ID     | Categoría      | Requisito                                                                    |
|--------|----------------|------------------------------------------------------------------------------|
| RNF-10 | Rendimiento    | Soportar pruebas de carga/estrés (p. ej. 200 hilos) con umbral configurable. |
| RNF-11 | Rendimiento    | Listados paginados para acotar el volumen por respuesta.                     |
| RNF-12 | Disponibilidad | Healthcheck de PostgreSQL y perfiles dev/prod.                               |
| RNF-13 | Disponibilidad | Endpoint de salud GET <code>/api/health</code> .                             |
| RNF-14 | Disponibilidad | Backups diarios ( <code>pg_dump</code> ) con retención de 30 días.           |
| RNF-15 | Disponibilidad | Cierre de ciclos y revalidación automáticos y programados.                   |
| RNF-16 | Usabilidad     | UI con componentes MUI responsive, labels, estados y mensajes guiados.       |
| RNF-17 | Usabilidad     | Experiencia diferenciada por rol.                                            |
| RNF-18 | Usabilidad     | Formularios con label/aria-label y landmark <code>main</code> .              |
| RNF-19 | Portabilidad   | Funcionar en Chrome, Firefox, Edge y Safari (escritorio).                    |
| RNF-20 | Portabilidad   | Desplegable en cualquier entorno con Docker.                                 |
| RNF-21 | Portabilidad   | Operar con APIs estándar del navegador sin polyfills.                        |
| RNF-22 | Mantenibilidad | Código en TypeScript (tipado estático) en cliente y servidor.                |
| RNF-23 | Mantenibilidad | Auditoría de acciones sensibles y bitácora de supervisor.                    |
| RNF-24 | Mantenibilidad | Identificador legible ( <code>uid</code> ) por entidad con prefijo.          |
| RNF-25 | Mantenibilidad | Batería de pruebas automatizadas y reportes de cobertura.                    |

## Modelo de casos de uso

### Diagramas de caso de uso

La Figura 2 presenta el diagrama general de casos de uso. La Tabla 6 lista el catálogo de casos de uso del sistema.





**Tabla 6***Catálogo de casos de uso*

| ID    | Caso de uso                                | Actor principal    |
|-------|--------------------------------------------|--------------------|
| CU-01 | Registrar estudiante                       | Estudiante         |
| CU-02 | Iniciar sesión                             | Todos              |
| CU-03 | Recuperar/restablecer contraseña           | Todos              |
| CU-04 | Configurar y verificar 2FA                 | Estudiante         |
| CU-05 | Gestionar estudiantes (CRUD)               | Administrador      |
| CU-06 | Importar estudiantes desde Excel           | Administrador      |
| CU-07 | Validar SISBÉN                             | Administrador      |
| CU-08 | Gestionar supervisores                     | Administrador      |
| CU-09 | Invitar supervisor / unirse por invitación | Admin / Supervisor |
| CU-10 | Asignar estudiantes a supervisor           | Administrador      |
| CU-11 | Subir pago y comprobantes                  | Estudiante         |
| CU-12 | Verificar pago                             | Administrador      |
| CU-13 | Calcular almuerzos por monto               | Autenticado        |
| CU-14 | Registrar almuerzo                         | Supervisor         |
| CU-15 | Consultar historial / asistencia del día   | Autenticado        |
| CU-16 | Consultar analítica del tablero            | Admin / Auditor    |
| CU-17 | Gestionar ciclos / cierre automático       | Admin / Sistema    |
| CU-18 | Calificar servicio                         | Estudiante         |
| CU-19 | Enviar y responder quejas                  | Estudiante / Admin |
| CU-20 | Gestionar noticias                         | Administrador      |
| CU-21 | Consultar auditoría                        | Admin / Auditor    |

**Descripción de casos de uso**

Se documentan los casos de uso más relevantes con la plantilla extendida (actor, precondiciones, flujo principal, flujos alternativos y postcondiciones). Las reglas y códigos de

respuesta provienen de los controladores reales y están verificados por las pruebas (Segunda Parte).

### ***CU-01 — Registrar estudiante***

**Actor:** Estudiante (no autenticado). **Endpoint:** POST /api/auth/register (multipart).

**Precondición:** el email no debe existir.

#### **Flujo principal**

(1) El estudiante diligencia el formulario y adjunta documentos. (2) Se valida la presencia de `archivoSisben`. (3) Se valida que el email no exista. (4) Se parsea `diasComedor`. (5) Se crea el usuario (rol student, contraseña cifrada) y el estudiante. (6) Respuesta 201.

#### **Alternativos**

2a. Falta el archivo SISBÉN → 400. 3a. Email ya registrado → 400.

#### **Postcondición**

Se crea un `User` (student) y un `Student` con `isValidatedSisben = false`.

### ***CU-02 — Iniciar sesión***

**Actor:** Todos. **Endpoint:** POST /api/auth/login.

#### **Flujo principal**

(1) Envío de email, contraseña y rol. (2) Validación de campos. (3) Búsqueda del usuario. (4) Verificación de contraseña (`bcrypt.compare`). (5) Validación de `isActive`, rol e `isAuthorized`. (6) Emisión de tokens y respuesta 200.

#### **Alternativos**

2a. Campos faltantes → 400. 3a. Usuario inexistente → 401. 4a. Contraseña incorrecta → 401. 5a. Inactivo → 401; rol no coincide → 403; no autorizado → 403. Excepción no controlada → 500 (sin filtrar el stack trace).

#### **Postcondición**

El cliente almacena los tokens para autenticar peticiones.

**CU-07 — Validar SISBÉN**

**Actor:** Administrador. **Endpoint:** POST /api/students/:id/validate-sisben.

**Flujo principal**

(1) Solicitud de validación. (2) Extracción del texto del documento SISBÉN (PDF con pdf-parse o imagen con OCR tesseract.js). (3) Normalización y cotejo de cédula, nombre y apellido, verificando que la cédula aparezca en el documento de identidad. (4) Si todo coincide, `isValidatedSisben = true` y respuesta 200.

**Alternativos**

1a. Estudiante no encontrado → 404. 3a. Inconsistencia → `validated = false` con el detalle de cada *mismatch*.

**Postcondición**

Se actualiza el estado de validación y `sisbenValidationDetails`.

**CU-11 — Subir pago y comprobantes**

**Actor:** Estudiante. **Endpoints:** POST /api/payments, POST /api/payments/upload, POST /api/payments/calculate.

**Flujo principal**

(1) Se ingresa el monto y se calcula `mealsIncluded = ⌊monto/2000⌋`. (2) Se adjunta el comprobante. (3) Se crea el pago con `isVerified = false` y respuesta 201.

**Alternativos**

1a. Monto < 2000 → 400. 1b. Estudiante no encontrado → 404. 2a. Sin archivo → 400.

**Postcondición**

Queda un pago pendiente de verificación.

**CU-14 — Registrar almuerzo (caso de uso central)**

**Actor:** Supervisor. **Endpoint:** POST /api/meals.

**Flujo principal**

(1) Búsqueda/escaneo del estudiante (QR/cédula). (2) Validación de existencia y estado

activo. (3) Validación de día autorizado. (4) Validación de no haber comido hoy. (5) Validación de disponibilidad de almuerzos. (6) Creación del `MealAttendance`, incremento de `mealsUsed` y creación de `SupervisorLog`. (7) Registro en `audit_logs` y respuesta 201.

### Alternativos

1a. No autenticado → 401. 2a. No encontrado → 404; inactivo → 400. 3a. Día no autorizado → 400. 4a. Ya comió hoy → 400. 5a. Sin almuerzos → 400.

### Postcondición

Disminuye la disponibilidad; quedan trazas en `meal_attendances`, `supervisor_logs` y `audit_logs`.

### *Otros casos de uso*

Por brevedad se resumen: **CU-12 Verificar pago** (PATCH `/api/payments/:id/verify`; 404 si no existe; tras verificar, los `mealsIncluded` cuentan como disponibles); **CU-09 Invitar/unir supervisor** (token válido 48 h; 400 ante campos faltantes, token inválido o email duplicado); **CU-18 Calificar** (1–5 estrellas; 401 sin autenticar, 404 si el estudiante no existe); **CU-19 Quejas** (anónimas o identificadas; respuesta del admin con 404 si no existe); **CU-21 Auditoría** (filtros validados por lista blanca; 400 ante filtros no permitidos).

## Modelo de diseño del sistema

### Diagrama de clases detallado

El modelo de dominio se implementa con Sequelize. La Figura 3 muestra el diagrama de clases. Todas las entidades incluyen `id` (UUID, PK), `uid` (identificador legible único con prefijo por tipo, generado en el hook `beforeValidate`), `createdAt` y `updatedAt`.

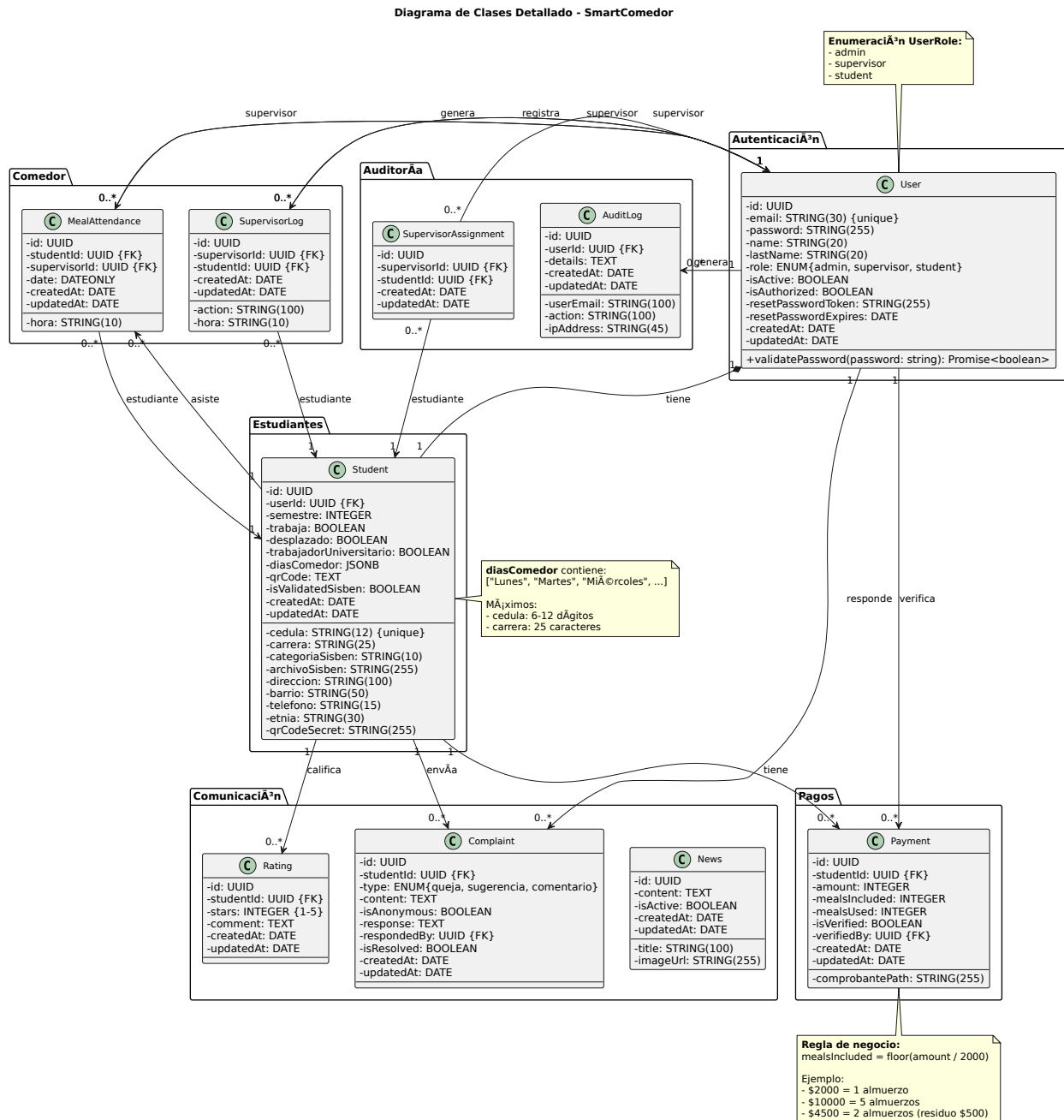
**Figura 3**

Diagrama de clases del dominio.

La Tabla 7 resume las entidades principales y sus atributos representativos.

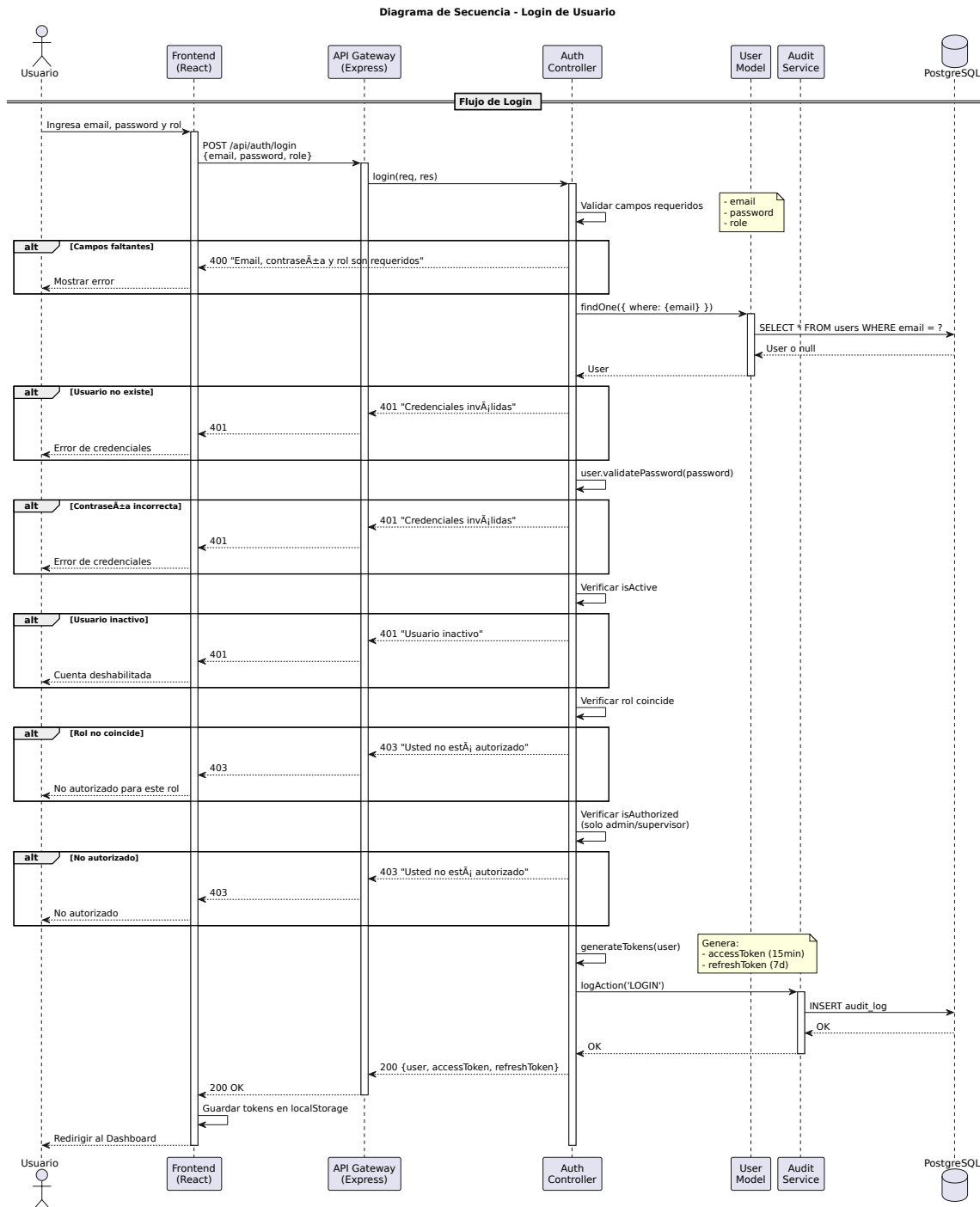
**Tabla 7***Entidades principales y atributos representativos*

| Entidad (tabla)                                            | Atributos representativos                                                                                                                                                                                                                                                               |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User ( <b>users</b> )                                      | email (único), password (hash), name, lastName, telefono, role, isActive, isAuthorized, diasAsignados, resetPasswordToken/Expires; método <code>validatePassword</code> .                                                                                                               |
| Student<br>( <b>students</b> )                             | userId (FK), cedula, carrera, semestre, categoriaSisben, archivoSisben, cedulaFrontalPath, horarioPdfPath, direccion, barrio, telefono, trabaja, etnia, desplazado, diasComedor, qrCode/qrCodeSecret, isValidatedSisben, sisbenValidationDetails, currentCycle, cycleRevalidationDueAt. |
| Payment<br>( <b>payments</b> )                             | studentId (FK), amount, mealsIncluded, mealsUsed, comprobantePath, universityReceiptPath, bankReceiptPath, isVerified, verifiedBy, verifiedAt.                                                                                                                                          |
| MealAttendance<br>( <b>meal_attendances</b> )              | studentId (FK), supervisorId (FK), date, hora.                                                                                                                                                                                                                                          |
| SupervisorAssignments<br>( <b>supervisor_assignments</b> ) | supervisorId (FK), studentId (FK) — relación N:M.                                                                                                                                                                                                                                       |
| SupervisorLog<br>( <b>supervisor_logs</b> )                | supervisorId (FK), studentId (FK), action, hora.                                                                                                                                                                                                                                        |
| Rating ( <b>ratings</b> )                                  | studentId (FK), stars (1–5), comment.                                                                                                                                                                                                                                                   |
| Complaint<br>( <b>complaints</b> )                         | studentId (FK, opcional), type (queja/sugerencia/comentario), content, isAnonymous, response, respondedBy, isResolved.                                                                                                                                                                  |
| News ( <b>news</b> )                                       | title, content, imageUrl, isActive.                                                                                                                                                                                                                                                     |
| Cycle ( <b>cycles</b> )                                    | name, startDate, endDate, status (Activo/Cerrado).                                                                                                                                                                                                                                      |
| AuditLog<br>( <b>audit_logs</b> )                          | userId, userEmail, action, details, ipAddress.                                                                                                                                                                                                                                          |

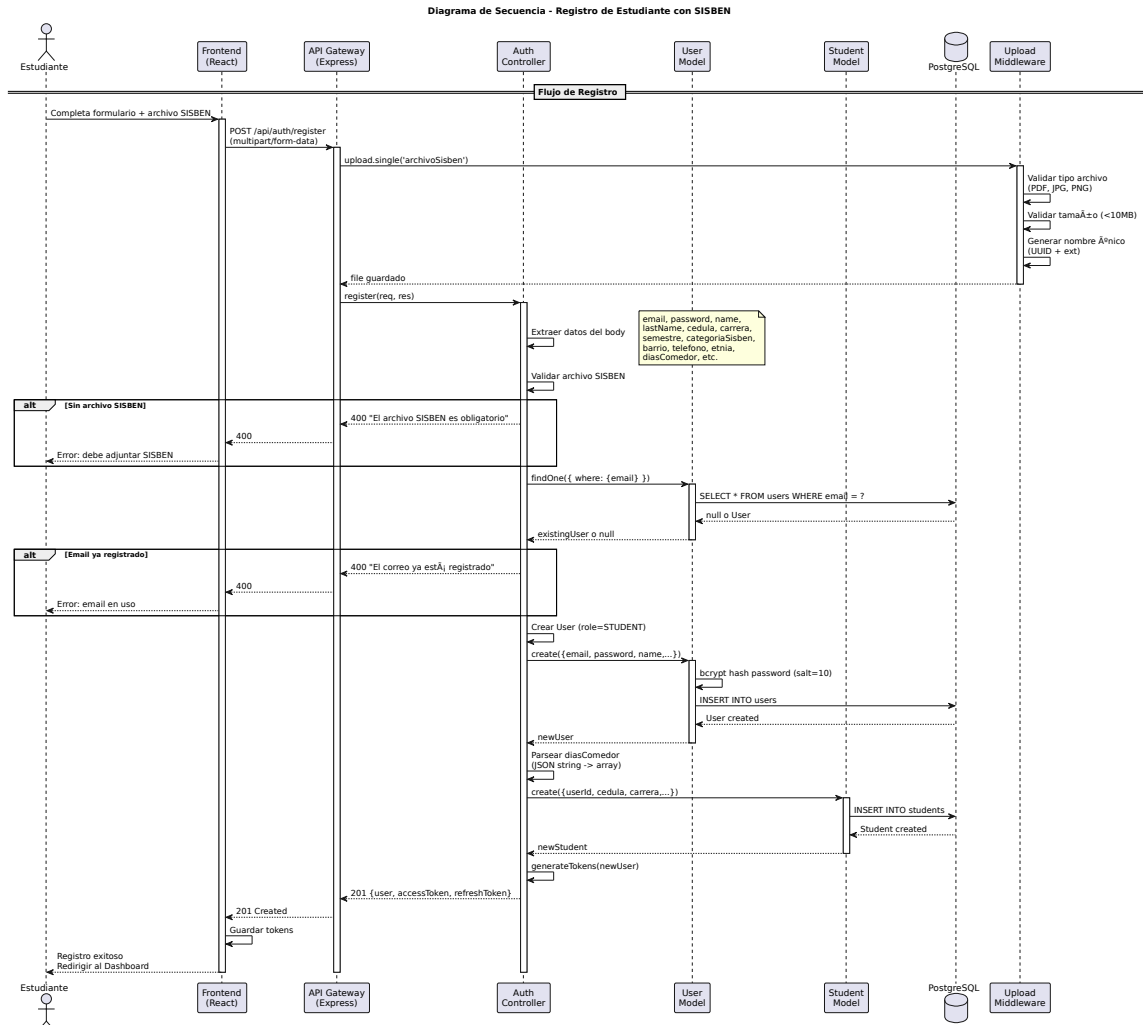
**Diagramas de secuencias**

Se documentan las secuencias de los flujos críticos: inicio de sesión (Figura 4), registro de estudiante (Figura 5), pago (Figura 6), registro de almuerzo (Figura 7) y validación SISBÉN (Figura 8).



**Figura 4**

*Diagrama de secuencia — Inicio de sesión.*

**Figura 5**

*Diagrama de secuencia — Registro de estudiante.*

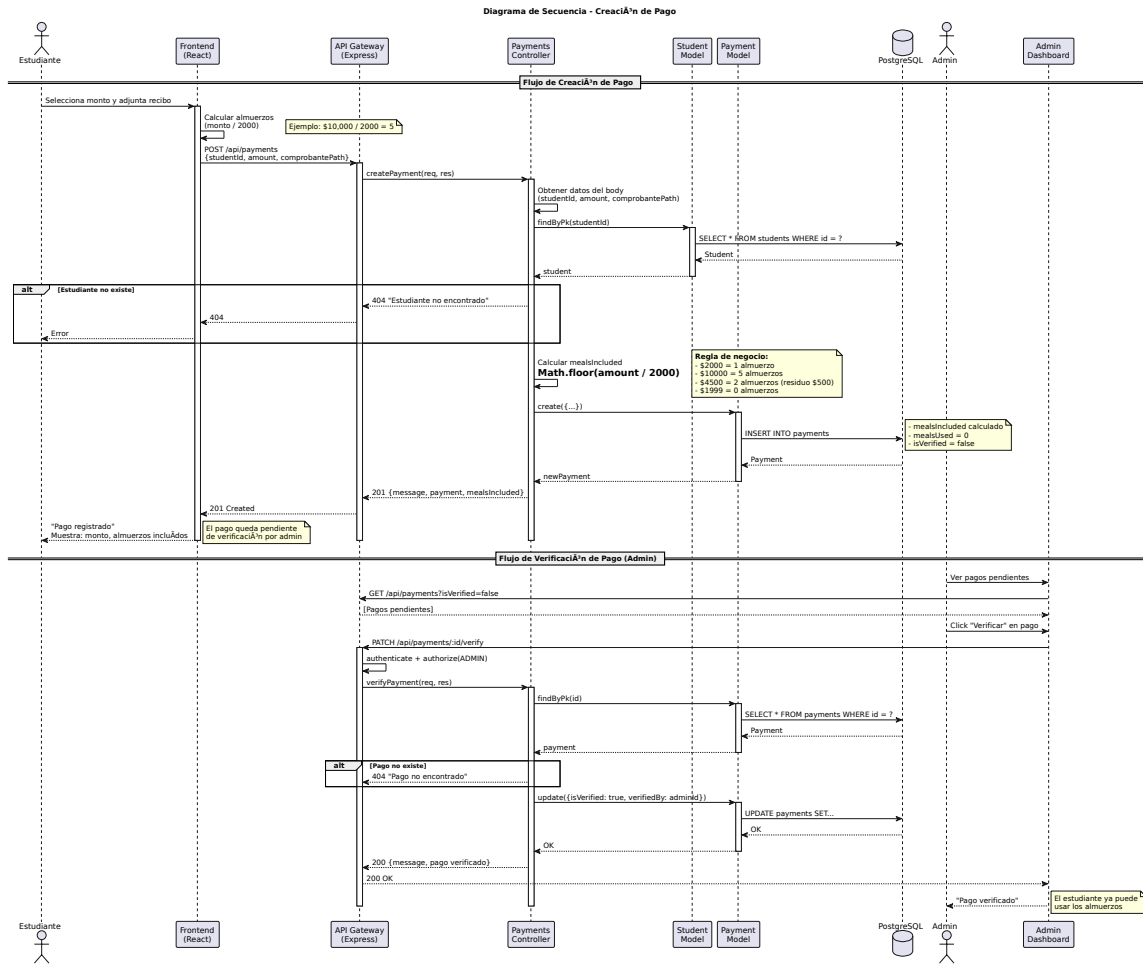


Figura 6

Diagrama de secuencia — Subida y verificación de pago.

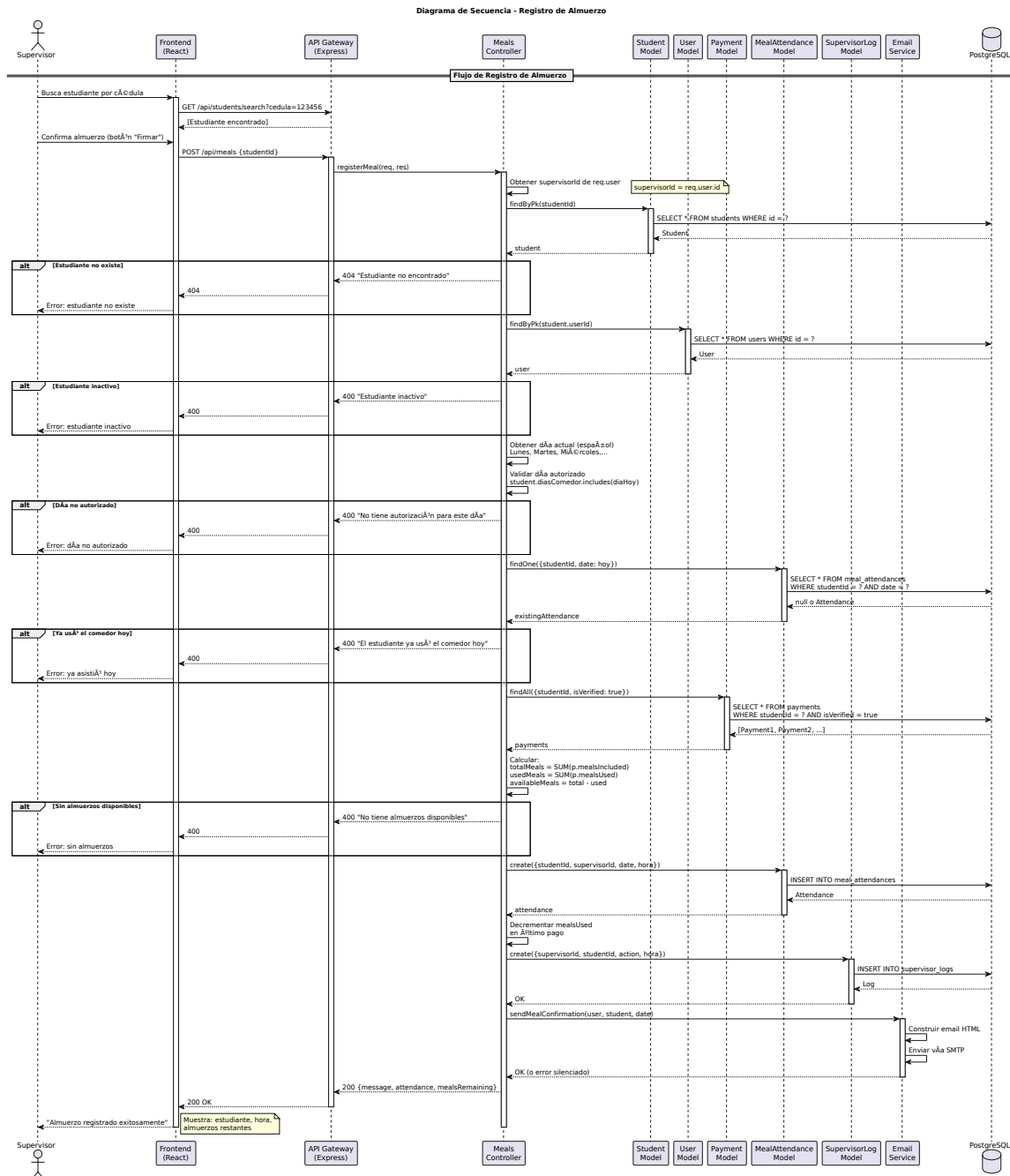
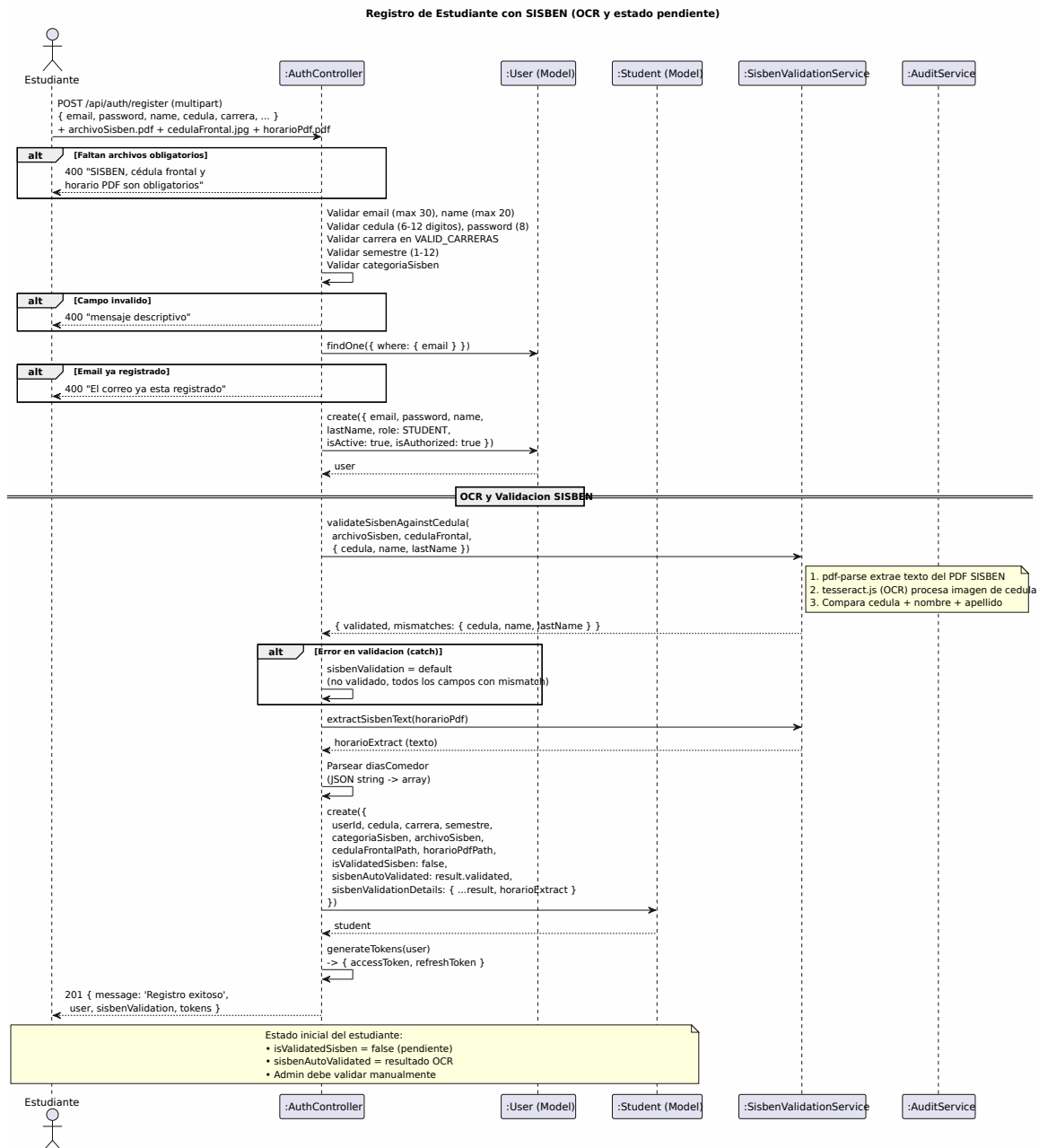


Figura 7

Diagrama de secuencia — Registro de almuerzo.

**Figura 8**

*Diagrama de secuencia — Validación SISBÉN.*

## Diagrama entidad-relación

La Figura 9 presenta el modelo entidad-relación. La Tabla 8 resume las cardinalidades.

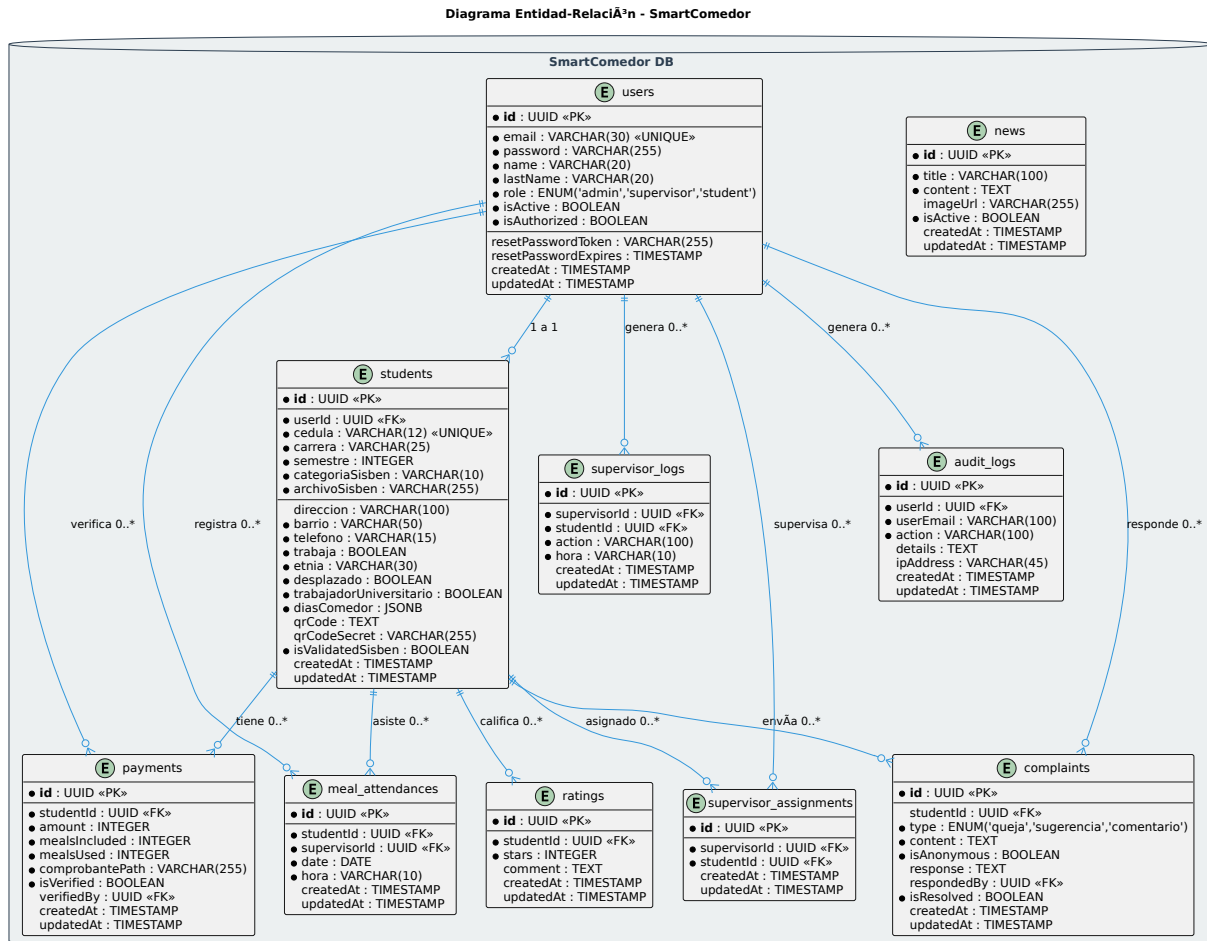
**Figura 9***Diagrama entidad-relación.*

Tabla 8

*Cardinalidades del modelo relacional*

| Relación                              | Cardinalidad                               |
|---------------------------------------|--------------------------------------------|
| USERS — STUDENTS                      | 1 : 1                                      |
| STUDENTS — PAYMENTS                   | 1 : N                                      |
| STUDENTS — MEAL_ATTENDANCES           | 1 : N                                      |
| USERS (supervisor) — MEAL_ATTENDANCES | 1 : N                                      |
| STUDENTS — RATINGS                    | 1 : N                                      |
| STUDENTS — COMPLAINTS                 | 1 : N (0..1 del lado estudiante, anónimas) |
| STUDENTS — SUPERVISOR_LOGS            | 1 : N                                      |
| USERS (supervisor) — STUDENTS         | N : M (vía SUPERVISOR_ASSIGNMENTS)         |
| USERS — AUDIT_LOGS                    | 1 : N                                      |

## Diagrama de componentes

La Figura 10 muestra la organización en componentes; la Tabla 9 describe sus responsabilidades.

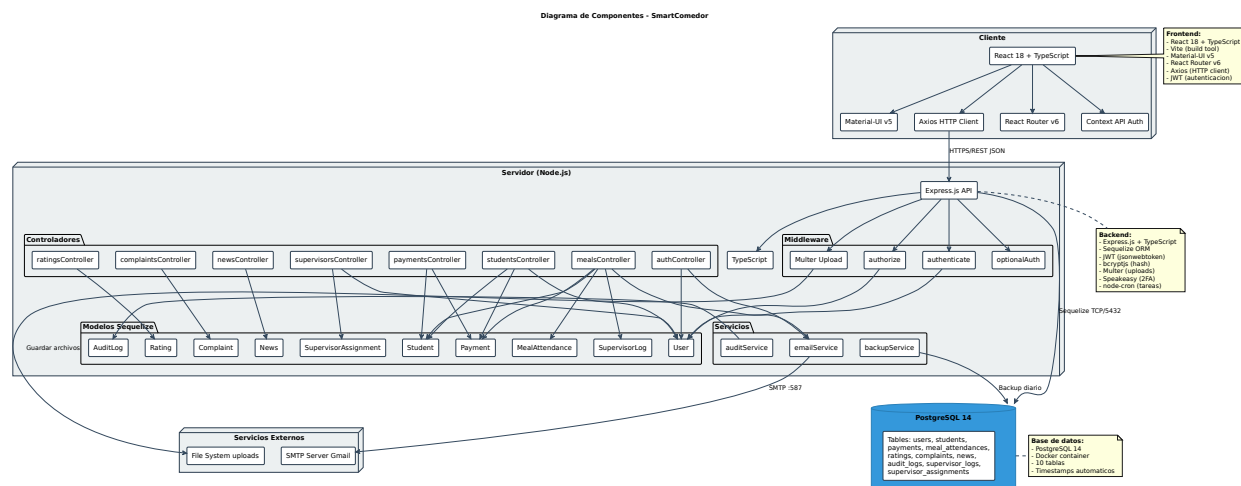


Figura 10

*Diagrama de componentes.*

**Tabla 9***Responsabilidades de los componentes*

| Componente              | Responsabilidad                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Frontend SPA            | Interfaz por rol, manejo de sesión (tokens), consumo de la API (axios), rutas protegidas.           |
| API Express             | Exposición de endpoints REST bajo <code>/api</code> , validación y orquestación de servicios.       |
| Middleware Auth         | <code>authenticate</code> (verifica JWT), <code>authorize</code> (rol), <code>optionalAuth</code> . |
| Middleware Upload       | Recepción de archivos con multer.                                                                   |
| Middleware EventAudit   | Registro automático de eventos HTTP en la auditoría.                                                |
| Controllers             | Lógica de negocio por entidad.                                                                      |
| EmailService            | Envío de correos (recuperación de contraseña, invitaciones).                                        |
| SisbenValidationService | Extracción de texto (PDF/OCR) y cotejo de elegibilidad.                                             |
| AuditService            | Persistencia de la bitácora de auditoría.                                                           |
| BackupService           | Respaldo periódico de la base de datos.                                                             |
| CycleAutomationService  | Cierre automático de ciclos y revalidación (cron).                                                  |
| Models (Sequelize)      | Mapeo objeto-relacional y asociaciones.                                                             |
| PostgreSQL              | Persistencia de datos.                                                                              |

## Producto del software

### Despliegue

El producto se despliega con Docker Compose. El Código 1 muestra los comandos principales.

```
# Variables de entorno
cp .env.example server/.env # completar JWT_SECRET, JWT_REFRESH_SECRET, SMTP_USER, SMTP_PASS

# Desarrollo (hot-reload): Frontend :5174, Backend :3002, PostgreSQL :5433
docker compose --profile dev up --build
```



```
# Produccion (builds + nginx): Frontend :80, Backend :3002
docker compose --profile prod up --build
```

Listing 1: Despliegue del producto

## Estructura del producto

```
SmartDinningRoom/
  client/      # Frontend React + Vite (SPA por rol)
  server/      # Backend Express + Sequelize (API REST /api)
  diagrams/    # Diagramas PlantUML + SVG
  docs/        # Documentacion, diagramas e informes
  tests/       # Pruebas de sistema (seguridad, rendimiento, portabilidad)
  docker-compose.yml
  .env.example
```

Listing 2: Estructura del proyecto

## Características destacadas

Seguridad por diseño (JWT con refresh, control de acceso por rol, 2FA TOTP, cifrado de contraseñas, validación de filtros por lista blanca y cabeceras de seguridad); antifraude (validación automática del SISBÉN por OCR/PDF e identificación con QR); trazabilidad total (`audit_logs` y `supervisor_logs`); automatización (importación masiva, cálculo de almuerzos, cierre de ciclos y backups programados); y operación 24/7 (healthchecks, perfiles dev/prod y backups con retención de 30 días).

# SEGUNDA PARTE. Pruebas del Software

## Introducción

Esta parte describe el proceso de verificación y validación (V&V) del sistema SmartComedor (IEEE, 2008). Las pruebas garantizan que el software cumple los requisitos funcionales y no funcionales y que las reglas de negocio críticas (elegibilidad SISBÉN, cálculo de

almuerzos, control de asistencia, control de acceso y auditoría) se comportan de forma correcta, segura y predecible. La estrategia combina varios niveles: pruebas unitarias (caja negra y caja blanca), pruebas de integración (incremental y basada en hilos), pruebas de sistema (seguridad, rendimiento, usabilidad y portabilidad) y pruebas de aceptación (Myers y col., 2011; Pressman & Maxim, 2014). Como resumen, la suite automatizada ejecuta **196 pruebas con 100 % de éxito** (10 suites) y una cobertura global aproximada del **80,4 %** de sentencias medida con Jest (Meta Open Source, 2024).

Planificación de las pruebas

Objetivos de las pruebas

Tabla 10

Objetivos de las pruebas

| ID     | Objetivo                                                                                                                                                |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| OBJ-01 | Verificar que cada requisito funcional (RF-01 . . . RF-45) está implementado y se comporta según lo especificado.                                       |
| OBJ-02 | Verificar las reglas de negocio críticas (cálculo de almuerzos, monto mínimo, un almuerzo por día, día autorizado, disponibilidad y validación SISBÉN). |
| OBJ-03 | Verificar el control de acceso (JWT, autorización por rol y manejo de cuentas inactivas/no autorizadas).                                                |
| OBJ-04 | Detectar defectos lo antes posible mediante pruebas automatizadas.                                                                                      |
| OBJ-05 | Validar atributos de calidad (seguridad, rendimiento, usabilidad, portabilidad).                                                                        |
| OBJ-06 | Asegurar una cobertura de código adecuada y trazable.                                                                                                   |
| OBJ-07 | Validar, con criterios de aceptación, la satisfacción de las necesidades del usuario.                                                                   |

Alcance de las pruebas

**Dentro del alcance:** controladores del backend; middleware de seguridad; servicios (validación SISBÉN, auditoría); flujos de integración entre módulos; atributos de calidad (cabeceras de seguridad, control de acceso, carga/estrés, portabilidad cross-browser y usabilidad/accesibilidad básica). **Fuera del alcance (o cubierto indirectamente):** pruebas

exhaustivas de componentes React; pruebas de proveedores externos (SMTP, infraestructura de BrowserStack); y *penetration testing* avanzado más allá del escaneo automatizado (OWASP ZAP) y JMeter (OWASP Foundation, 2021).

## Estrategias de pruebas

La Tabla 11 resume el mapeo de niveles, técnicas y herramientas.

**Tabla 11**

*Estrategias de pruebas por nivel*

| Nivel                  | Técnica                                                          | Herramienta / ubicación                                   |
|------------------------|------------------------------------------------------------------|-----------------------------------------------------------|
| Unitario               | Caja negra (equivalencia, valores límite) + caja blanca (camino) | Jest + ts-jest<br>( <code>server/src/tests/unit/</code> ) |
| Integración            | Incremental (bottom-up)                                          | Jest<br>( <code>integration/incremental.test.ts</code> )  |
| Integración            | Basada en hilos (por actor)                                      | Jest<br>( <code>integration/threads.test.ts</code> )      |
| Sistema — Seguridad    | Cabeceras, control de acceso, escaneo                            | Playwright + OWASP ZAP + JMeter                           |
| Sistema — Rendimiento  | Carga y estrés                                                   | Apache JMeter<br>( <code>tests/security/jmeter/</code> )  |
| Sistema — Portabilidad | Cross-browser                                                    | Playwright + BrowserStack                                 |
| Sistema — Usabilidad   | Inspección heurística + accesibilidad                            | Playwright + revisión MUI                                 |
| Aceptación             | Escenarios de usuario (Gherkin)                                  | Manual / Playwright                                       |

Las técnicas de diseño empleadas son: **particiones de equivalencia** (clases válidas e inválidas), **análisis de valores límite** (fronteras de rangos), **cobertura de caminos** (grafo de flujo) y **pruebas basadas en estado** (Beizer, 1990; Myers y col., 2011).

## Ambiente de pruebas

La Tabla 12 describe el ambiente. Las pruebas unitarias y de integración emplean una base de datos simulada en memoria (mocks), lo que permite ejecutarlas sin PostgreSQL real, de forma determinista y veloz.

**Tabla 12**

*Ambiente de pruebas*

| Elemento                    | Versión / detalle                                         |
|-----------------------------|-----------------------------------------------------------|
| Node.js                     | 20+ (probado con v22)                                     |
| Framework de pruebas        | Jest 29 + ts-jest 29                                      |
| Lenguaje                    | TypeScript 5                                              |
| BD en unitarias/integración | Mock en memoria (sin PostgreSQL real)                     |
| Pruebas de sistema          | Docker (JMeter), Playwright 1.44, BrowserStack, OWASP ZAP |

```
cd server
npm test           # toda la suite
npm run test:unit  # solo unitarias
npm run test:integration # solo integracion
npm run test:coverage # con reporte de cobertura
```

Listing 3: Scripts de ejecución (backend)

## Pruebas unitarias

### Análisis de las pruebas

Las pruebas unitarias aíslan la unidad bajo prueba (controlador, middleware o servicio) mediante mocks de los modelos Sequelize, de `jsonwebtoken`, de `bcrypt`, de `nodemailer` y del sistema de archivos. Se priorizaron las unidades de mayor riesgo: `authController` (seguridad de acceso, tokens, recuperación de contraseña, 2FA), `middleware/auth` (control de acceso), `paymentsController` (cálculo monetario), `mealsController` (reglas de asistencia), `studentsController` (CRUD, importación, SISBÉN) y los servicios de validación. La Tabla 13

presenta la distribución de casos por suite.

**Tabla 13**

*Distribución de casos por suite*

| Suite (archivo)                      | Casos it()                                       |
|--------------------------------------|--------------------------------------------------|
| unit/authController.test.ts          | 59                                               |
| unit/studentsController.test.ts      | 28                                               |
| unit/paymentsController.test.ts      | 21                                               |
| unit/supervisorsController.test.ts   | 19                                               |
| unit/ratingsComplaintsNews.test.ts   | 17                                               |
| unit/middleware.test.ts              | 15                                               |
| unit/mealsController.test.ts         | 13                                               |
| unit/sisbenValidationService.test.ts | 6                                                |
| integration/incremental.test.ts      | 16                                               |
| integration/threads.test.ts          | 8                                                |
| <b>Total</b>                         | <b>202 declarados (196 ejecutados/aprobados)</b> |

## Diseño de casos de prueba

### *Caja negra — Clases de equivalencia (Login)*

La Tabla 14 muestra los casos de equivalencia del inicio de sesión.

**Tabla 14**

*Clases de equivalencia — Login*

| Caso         | Entrada               | Clase             | Esperado     |
|--------------|-----------------------|-------------------|--------------|
| TC-LOGIN-001 | admin válido          | Válida            | 200 + tokens |
| TC-LOGIN-002 | estudiante válido     | Válida            | 200 + tokens |
| TC-LOGIN-003 | supervisor autorizado | Válida            | 200 + tokens |
| TC-LOGIN-004 | sin email             | Inválida (campos) | 400          |
| TC-LOGIN-005 | sin password          | Inválida (campos) | 400          |

| Caso         | Entrada                  | Clase                   | Esperado |
|--------------|--------------------------|-------------------------|----------|
| TC-LOGIN-006 | sin role                 | Inválida (campos)       | 400      |
| TC-LOGIN-007 | usuario inexistente      | Inválida (credenciales) | 401      |
| TC-LOGIN-008 | contraseña incorrecta    | Inválida (credenciales) | 401      |
| TC-LOGIN-009 | usuario inactivo         | Inválida (estado)       | 401      |
| TC-LOGIN-010 | rol no coincide          | Inválida (rol)          | 403      |
| TC-LOGIN-011 | admin no autorizado      | Inválida (autorización) | 403      |
| TC-LOGIN-012 | supervisor no autorizado | Inválida (autorización) | 403      |
| TC-LOGIN-013 | excepción del servidor   | Error interno           | 500      |

### *Caja blanca — Cobertura de caminos (login)*

El método `login` tiene un grafo de flujo con 7 caminos independientes; su complejidad ciclomática es 7 (McCabe, 1976). La Tabla 15 lista los caminos cubiertos.

**Tabla 15**

*Cobertura de caminos del flujo de login*

| Camino | Descripción                                   | Esperado |
|--------|-----------------------------------------------|----------|
| 1      | Campos faltantes                              | 400      |
| 2      | Campos OK → usuario no existe                 | 401      |
| 3      | Campos OK → contraseña incorrecta             | 401      |
| 4      | Campos OK → pass OK → inactivo                | 401      |
| 5      | Campos OK → pass OK → activo → rol incorrecto | 403      |
| 6      | Camino completo hasta no autorizado           | 403      |
| 7      | Camino completo exitoso                       | 200      |

***Caja negra — Valores límite (registro y montos)***

La Tabla 16 resume el análisis de valores límite.

**Tabla 16**

*Análisis de valores límite*

| Caso          | Atributo | Valor            | Resultado                 |
|---------------|----------|------------------|---------------------------|
| VL-EMAIL-001  | email    | 30 caracteres    | válido                    |
| VL-CEDULA-001 | cédula   | 6 dígitos (mín)  | válido                    |
| VL-CEDULA-002 | cédula   | 12 dígitos (máx) | válido                    |
| VL-CEDULA-003 | cédula   | 5 dígitos        | inválido                  |
| VL-NOMBRE-001 | nombre   | 20 caracteres    | válido                    |
| VL-NOMBRE-002 | nombre   | 21 caracteres    | inválido                  |
| VL-MONTO-001  | monto    | 2000             | 1 almuerzo                |
| VL-MONTO-002  | monto    | 1999             | 0 almuerzos               |
| VL-MONTO-003  | monto    | 10000            | 5 almuerzos               |
| VL-MONTO-004  | monto    | 4500             | 2 almuerzos (residuo 500) |
| VL-CALC-002   | calcular | 1999             | 400 (bajo el mínimo)      |
| VL-CALC-005   | calcular | 10000            | 5 almuerzos               |
| VL-CALC-006   | calcular | 24000            | 12 almuerzos              |
| VL-CALC-007   | calcular | $\leq 0$         | 400                       |

***Registro de almuerzo y middleware***

La Tabla 17 resume casos del registro de almuerzo y del middleware de seguridad.

**Tabla 17**

*Casos de registro de almuerzo y middleware de seguridad*

| Caso        | Escenario                          | Esperado |
|-------------|------------------------------------|----------|
| TC-MEAL-001 | Registro exitoso (camino completo) | 201      |

| Caso           | Escenario                                          | Esperado                      |
|----------------|----------------------------------------------------|-------------------------------|
| TC-MEAL-002    | Supervisor no autenticado                          | 401                           |
| TC-MEAL-003    | Estudiante no encontrado                           | 404                           |
| TC-MEAL-004    | Estudiante inactivo                                | 400                           |
| TC-MEAL-005    | Día no autorizado                                  | 400                           |
| TC-MEAL-006    | Ya usó el comedor hoy                              | 400                           |
| TC-MEAL-007    | Sin almuerzos disponibles                          | 400                           |
| TC-MEAL-008    | Actualiza <code>mealsUsed</code> en el último pago | OK                            |
| TC-MEAL-009    | Registra <code>SupervisorLog</code>                | OK                            |
| TC-AUTH-MW-002 | Sin header <code>Authorization</code>              | 401                           |
| TC-AUTH-MW-004 | Token JWT inválido                                 | 401                           |
| TC-AUTH-MW-006 | Usuario inactivo                                   | 401                           |
| TC-AUTHZ-003   | Rol no permitido                                   | 403                           |
| TC-OPT-003     | Token inválido ( <code>optionalAuth</code> )       | <code>next()</code> sin error |

El catálogo completo de los 202 casos (refresh token, perfil, recuperación de contraseña, 2FA, estudiantes, supervisores, pagos, ratings, quejas y noticias) está implementado en `server/src/tests/unit/` con identificadores `TC-*/VL-*`.

## Ejecución y evaluación de las pruebas

### *Resultado de ejecución*

La ejecución de `npm run test:coverage` produjo el resultado del Código 4.

```
Test Suites: 10 passed, 10 total
Tests:      196 passed, 196 total
Snapshots:  0 total
Time:       ~4.7 s
```

Listing 4: Resultado de ejecución de la suite

### *Cobertura de código*

La Tabla 18 presenta la cobertura por ámbito (Jest/Istanbul).



**Tabla 18***Cobertura de código (Jest/Istanbul)*

| Ámbito                     | % Sent.      | % Ramas      | % Func.      | % Líneas     |
|----------------------------|--------------|--------------|--------------|--------------|
| <b>Total</b>               | <b>80,44</b> | <b>70,25</b> | <b>76,41</b> | <b>79,91</b> |
| constants                  | 100          | 100          | 100          | 100          |
| middleware (auth.ts)       | 97,50        | 91,66        | 100          | 97,29        |
| services                   | 96,66        | 78,94        | 87,50        | 96,36        |
| sisbenValidationService.ts | 100          | 93,33        | 100          | 100          |
| auditService.ts            | 83,33        | 25,00        | 50,00        | 80,00        |
| controllers (global)       | 78,47        | 67,75        | 74,19        | 77,90        |
| authController.ts          | 83,33        | 81,19        | 84,61        | 82,93        |
| ratingsController.ts       | 85,00        | 66,66        | 100          | 82,85        |
| complaintsController.ts    | 80,00        | 60,00        | 100          | 82,85        |
| supervisorsController.ts   | 79,54        | 80,00        | 78,57        | 77,68        |
| paymentsController.ts      | 78,94        | 70,45        | 66,66        | 77,27        |
| studentsController.ts      | 78,50        | 62,02        | 78,26        | 77,34        |
| newsController.ts          | 77,55        | 66,66        | 100          | 75,00        |
| mealsController.ts         | 63,00        | 31,81        | 31,25        | 65,11        |

***Evaluación***

La tasa de éxito es del 100 % (196/196); no se detectaron defectos abiertos en la suite automatizada. La lógica de seguridad (`middleware/auth.ts`) y la validación SISBÉN alcanzan cobertura muy alta ( $\geq 97\%$  y 100 % de sentencias), apropiada por ser componentes críticos. Las áreas con menor cobertura —`mealsController.ts` (analítica del tablero) y `auditService.ts` (manejo de error del logging)— son oportunidades de mejora: se recomienda añadir casos para `getDashboardAnalytics` y para los caminos de error de `auditService` a fin de elevar la cobertura de ramas por encima del 80 %.

## Pruebas de integración

Las pruebas de integración verifican que los módulos colaboran correctamente, con dos estrategias complementarias y una base de datos simulada en memoria que reproduce las asociaciones reales.

### Estrategia de pruebas incrementales

#### *Descripción*

Implementada en `integration/incremental.test.ts`, integra los módulos de abajo hacia arriba (bottom-up): (1) Auth (base), (2) Students (usa Auth), (3) Payments (usa Students) y (4) Meals (usa Students + Payments). Cada nivel se prueba solo cuando el inferior ya está integrado, de modo que un fallo se localiza en el módulo recién incorporado.

#### *Diseño y ejecución de casos*

La Tabla 19 resume los casos. Resultado: PASS (16 casos).

**Tabla 19**

*Pruebas de integración incrementales*

| Caso         | Módulos             | Escenario                                        | Esperado         |
|--------------|---------------------|--------------------------------------------------|------------------|
| INT-AUTH-001 | Auth                | Register → Login → Profile                       | Perfil correcto  |
| INT-AUTH-002 | Auth                | Login con credenciales incorrectas               | Falla controlada |
| INT-AUTH-003 | Auth                | Admin no autorizado no puede loguearse           | 403              |
| INT-STU-001  | Auth + Students     | Crear usuario/estudiante y obtener con almuerzos | OK               |
| INT-STU-002  | Students            | Validar SISBÉN actualiza flag en student         | flag = true      |
| INT-STU-003  | Auth + Students     | Deshabilitar usuario deshabilita estudiante      | inactivo         |
| INT-PAY-001  | Students + Payments | Crear pago y calcular almuerzos                  | OK               |

| Caso         | Módulos                        | Escenario                                       | Esperado        |
|--------------|--------------------------------|-------------------------------------------------|-----------------|
| INT-PAY-002  | Payments                       | Múltiples pagos acumulan al-<br>muerzos         | suma correcta   |
| INT-PAY-003  | Payments                       | Verificar pago lo marca verifica-<br>do         | isVerified=true |
| INT-MEAL-001 | Students + Payments +<br>Meals | Registrar almuerzo decrementa<br>disponibilidad | -1              |
| INT-MEAL-002 | Meals                          | No registrar si ya comió hoy                    | rechazo         |
| INT-MEAL-003 | Meals                          | No registrar sin almuerzos                      | rechazo         |
| INT-MEAL-004 | Meals                          | Registrar almuerzo crea<br>SupervisorLog        | log creado      |

**Evaluación.** Se confirma que las interfaces entre módulos funcionan: el flag de SISBÉN se propaga al estudiante, la verificación de pagos habilita almuerzos disponibles y el registro de almuerzo actualiza la disponibilidad y la bitácora. La integración bottom-up valida que la regla acumulativa de almuerzos (varios pagos) y la regla de consumo (un almuerzo por día) operan en conjunto.

## Estrategia de pruebas basadas en hilos

### *Descripción*

Implementada en `integration/threads.test.ts`, cada “hilo” prueba la secuencia completa de interacciones de un actor de principio a fin: **Hilo 1 (Estudiante)** Registro → Login → Perfil → Pago → Almuerzos → Calificar → Noticias → Queja; **Hilo 2 (Supervisor)** Login → Buscar estudiante → Registrar almuerzo → Historial; **Hilo 3 (Administrador)** Login → Crear supervisor → Asignar → Verificar pago → Validar SISBÉN → Reportes.

### *Diseño y ejecución de casos*

La Tabla 20 resume los casos. Resultado: PASS (8 casos).

Tabla 20

*Pruebas de integración basadas en hilos*

| Caso   | Hilo          | Escenario                                                            | Esperado   |
|--------|---------------|----------------------------------------------------------------------|------------|
| H1-001 | Estudiante    | Flujo completo desde registro hasta envío de queja                   | Pasos OK   |
| H1-002 | Estudiante    | No puede registrar almuerzo sin pagos                                | rechazo    |
| H2-001 | Supervisor    | Supervisor registra almuerzo de estudiante                           | registrado |
| H3-xxx | Administrador | Login → crear supervisor → asignar → verificar pago → validar SISBÉN | flujo OK   |

**Evaluación.** Las pruebas por hilo validan los recorridos de usuario reales end-to-end a nivel de lógica de negocio, confirmando que la composición de operaciones de cada rol produce el estado esperado. Mientras las incrementales validan interfaces entre capas, las de hilos validan flujos completos por actor.

Pruebas de sistema

Las pruebas de sistema evalúan atributos de calidad sobre el sistema desplegado, según el modelo ISO/IEC 25010 (International Organization for Standardization, 2011).

Pruebas de seguridad

*Diseño*

Se diseñaron pruebas automatizadas en tres frentes: API y frontend con Playwright (`security.test.ts`); escaneo dinámico con OWASP ZAP (DAST); e inyección/fuzzing con JMeter (`security-plan.jmx` y `sql-payloads.csv`) (Apache Software Foundation, 2024; Microsoft, 2024; OWASP Foundation, 2021). La Tabla 21 resume casos representativos.

**Tabla 21***Casos de prueba de seguridad (Playwright)*

| Grupo     | Caso                                  | Verificación         |
|-----------|---------------------------------------|----------------------|
| Cabeceras | X-Powered-By no expuesto              | No revela tecnología |
| Cabeceras | X-Content-Type-Options: nosniff       | Evita MIME sniffing  |
| Cabeceras | Protección anti-clickjacking          | X-Frame-Options/CSP  |
| Acceso    | GET /api/students sin token           | 401                  |
| Acceso    | GET /api/audit sin token              | 401                  |
| Acceso    | Login con credenciales inválidas      | 401                  |
| Errores   | Login no filtra stack traces          | Mensaje genérico     |
| Rutas     | Endpoint inexistente                  | 404 (no 200)         |
| Frontend  | Redirige a login si no autenticado    | Redirección          |
| Frontend  | Password con autocomplete='off'       | No autocompleta      |
| Frontend  | No expone tokens en la URL tras login | URL limpia           |
| Frontend  | Bundle no expone variables sensibles  | Sin secretos         |
| Frontend  | Cookies de sesión con HttpOnly        | No accesibles por JS |

***Ejecución y evaluación***

El diseño valida los requisitos RNF-01... RNF-08. El control de acceso por token y rol queda corroborado a nivel unitario (TC-AUTH-MW-\*, TC-AUTHZ-\*) y de sistema (rechazos 401 en endpoints protegidos). La política de no filtrar stack traces y el mensaje genérico en recuperación de contraseña (TC-FORGOT-003) reducen la fuga de información. Los filtros de auditoría usan listas blancas (método, código, ruta, rol), mitigando la inyección por parámetros. Para inyección SQL, el uso de Sequelize con consultas parametrizadas y el plan JMeter con `sql-payloads.csv` permiten comprobar que los payloads no alteran el comportamiento ni exponen datos. *Nota:* estas pruebas requieren la app desplegada (frontend en :5174, backend en :3002) y, para BrowserStack/ZAP, credenciales y túnel local.

Pruebas de rendimiento

Diseño

Implementadas con Apache JMeter en Docker (Apache Software Foundation, 2024): `load-test.jmx` (carga sostenida), `stress-test.jmx` (punto de quiebre) y `security-plan.jmx` (aserciones de seguridad). La Tabla 22 muestra los parámetros por defecto del script `run-load-stress.sh`.

Tabla 22  
*Parámetros por defecto de la prueba de carga/estrés*

| Parámetro          | Valor | Significado                             |
|--------------------|-------|-----------------------------------------|
| LOAD_THREADS       | 200   | Usuarios virtuales concurrentes         |
| LOAD_RAMP_SECS     | 60    | Tiempo de rampa de subida               |
| LOAD_DURATION_SECS | 300   | Duración de la prueba                   |
| THRESHOLD_MS       | 500   | Umbral de tiempo de respuesta aceptable |

Ejecución y evaluación

El objetivo (RNF-09/RNF-10) es que la autenticación y las consultas críticas respondan dentro del umbral (objetivo < 3 s; umbral estricto configurado en 500 ms). El reporte HTML de JMeter entrega métricas de tiempo de respuesta (media, percentiles), *throughput* y tasa de error. El criterio de aceptación es una tasa de error  $\approx 0\%$  y un percentil 95 por debajo del umbral bajo la carga objetivo. La paginación de los listados (RNF-11) acota el tamaño de respuesta y favorece el rendimiento bajo alto volumen de datos.

Pruebas de usabilidad

Diseño

La usabilidad se evaluó mediante inspección heurística (Nielsen, 1994) y verificaciones automatizadas de accesibilidad básica, alineadas con la regla de diseño Material Design 3 / MUI del proyecto (Google, 2024). La Tabla 23 mapea heurísticas y verificaciones.

**Tabla 23***Heurísticas de usabilidad y verificación*

| Heurística / criterio              | Verificación                                                                         |
|------------------------------------|--------------------------------------------------------------------------------------|
| Visibilidad del estado del sistema | Estados de carga/disabled/success/error<br>( <code>FeedbackState</code> )            |
| Correspondencia con el mundo real  | Copys en español y terminología del dominio                                          |
| Prevención de errores              | Validación de formularios, labels visibles, campos requeridos                        |
| Reconocer antes que recordar       | Navegación por rol; <code>Layout</code> y <code>DataSectionCard</code> reutilizables |
| Flexibilidad y eficiencia          | Asistente virtual, búsqueda y paginación                                             |
| Diseño estético y minimalista      | Componentes MUI responsive, jerarquía tipográfica                                    |
| Accesibilidad                      | label/aria-label en inputs, landmark <code>main</code> , foco visible                |

### ***Ejecución y evaluación***

La interfaz por rol y los componentes reutilizables garantizan consistencia y reducen la carga cognitiva (RNF-16/RNF-17/RNF-18). Las verificaciones de accesibilidad básica (labels, landmark, foco) pasan en el formulario de login. Se recomienda complementar con pruebas de usabilidad con usuarios reales (tareas cronometradas y cuestionario SUS) para obtener una métrica cuantitativa de satisfacción.

### **Pruebas de portabilidad**

#### ***Diseño***

Implementadas con Playwright sobre BrowserStack (Microsoft, 2024). La matriz de navegadores/plataformas (definida en `browserstack.yml`) cubre Chrome (Windows 11), Firefox (macOS Sonoma), Edge (Windows 10) y Safari (macOS Ventura). Las suites verifican: carga y renderizado (sin errores JS, título, viewport, recursos críticos); navegación y routing (`/login`, ruta protegida, History API, 404 por la SPA); formulario de login; APIs del navegador (localStorage, fetch, Promise, CSS variables, Flexbox); conectividad API (`/api/health`, CORS);

y accesibilidad básica.

### ***Ejecución y evaluación***

Valida RNF-19/RNF-20/RNF-21: el frontend funciona en los cuatro navegadores principales sin polyfills (APIs ES2015+ disponibles); la SPA maneja routing, History API y rutas 404 de forma consistente; y la portabilidad de despliegue se garantiza con Docker Compose (independiente del SO anfitrión). *Nota:* requiere BROWSERSTACK\_USERNAME/ACCESS\_KEY y el túnel BrowserStack Local activo.

## **Pruebas de aceptación**

### **Diseño de casos de prueba**

Las pruebas de aceptación validan, desde la perspectiva del usuario, que los flujos de negocio completos satisfacen las necesidades reales, expresadas como criterios en formato Gherkin (Dado–Cuando–Entonces).

#### **CA-01 Registro y validación de estudiante elegible**

*Dado* un estudiante con categoría SISBÉN válida y sus documentos, *cuando* se registra y el administrador valida su SISBÉN, *entonces* su cuenta queda creada, su estado de validación pasa a “validado” y puede iniciar sesión. *Criterio:* registro 201; validación cotejada; login 200.

#### **CA-02 Recarga y verificación de almuerzos**

*Dado* un estudiante autenticado, *cuando* sube un comprobante por \$10.000 y el administrador lo verifica, *entonces* se le acreditan 5 almuerzos. *Criterio:*  $\lfloor 10000/2000 \rfloor = 5$ ; disponibles = 5 tras verificar.

#### **CA-03 Registro de asistencia**

*Dado* un supervisor autorizado y un estudiante con almuerzos en un día autorizado, *cuando* registra su almuerzo, *entonces* se descuenta un almuerzo, queda la asistencia con fecha/hora y se crea el registro de bitácora. *Criterio:* 201; disponibilidad –1; SupervisorLog y AuditLog creados.

#### **CA-04 Prevención de doble consumo**

*Dado* un estudiante que ya comió hoy, *cuando* el supervisor intenta registrar otro almuerzo,



*entonces* el sistema lo rechaza. *Criterio:* 400 “ya usó el comedor hoy”.

### CA-05 Retroalimentación

*Dado* un estudiante autenticado, *cuando* califica con 4 estrellas y envía una sugerencia anónima, *entonces* la calificación se contabiliza y la sugerencia queda sin asociar su identidad.

*Criterio:* rating creado; promedio actualizado; queja con `isAnonymous = true`.

### CA-06 Control de acceso por rol

*Dado* un estudiante autenticado, *cuando* intenta acceder a la auditoría, *entonces* el sistema deniega el acceso. *Criterio:* 403 para rol no permitido; 200 solo para admin/auditor.

### CA-07 Auditoría y trazabilidad

*Dado* un administrador o auditor, *cuando* consulta la auditoría filtrando por fecha y ruta, *entonces* obtiene los eventos paginados y los filtros inválidos son rechazados. *Criterio:* 200 con paginación; 400 ante filtros fuera de lista blanca.

## Ejecución y evaluación de la prueba

La Tabla 24 relaciona cada criterio de aceptación con las pruebas que lo respaldan.

**Tabla 24**

*Trazabilidad de las pruebas de aceptación*

| Caso  | Soporte de verificación                           | Resultado |
|-------|---------------------------------------------------|-----------|
| CA-01 | INT-STU-002, TC-SISBEN-001, INT-AUTH-001          | Aceptado  |
| CA-02 | VL-CALC-005, INT-PAY-001/003                      | Aceptado  |
| CA-03 | TC-MEAL-001/008/009, INT-MEAL-001/004             | Aceptado  |
| CA-04 | TC-MEAL-006, INT-MEAL-002                         | Aceptado  |
| CA-05 | TC-RAT-001, TC-COMP-001, TC-AVG-001               | Aceptado  |
| CA-06 | TC-AUTHZ-003, pruebas de seguridad de acceso      | Aceptado  |
| CA-07 | RF-45, validación de listas blancas (auditRoutes) | Aceptado  |

Todos los criterios quedan respaldados por pruebas automatizadas (unitarias y de integración) y por las pruebas de sistema. Se concluye que el sistema satisface los flujos de negocio esperados de

los cuatro actores y cumple las reglas críticas de elegibilidad, recarga, consumo, retroalimentación, control de acceso y trazabilidad.

### Conclusiones

1. **Calidad funcional verificada.** La suite automatizada ejecuta 196 pruebas con 100 % de éxito (10 suites), cubriendo controladores, middleware, servicios y flujos de integración. Las reglas de negocio críticas están probadas con técnicas de caja negra y caja blanca.
2. **Cobertura sólida con áreas de mejora.** La cobertura global es de  $\sim 80,4\%$  de sentencias y  $70,3\%$  de ramas. Los componentes críticos para la seguridad (`middleware/auth`) y la elegibilidad (`sisbenValidationService`) alcanzan cobertura muy alta. Se recomienda elevar la cobertura de ramas de `mealsController` y `auditService`.
3. **Estrategias de integración complementarias.** La estrategia incremental (bottom-up) valida las interfaces entre capas y la basada en hilos valida los recorridos completos por actor; ambas pasan al 100 %.
4. **Atributos de calidad cubiertos.** Se dispone de planes ejecutables para seguridad (`Playwright` + `OWASP ZAP` + `JMeter`), rendimiento (carga/estrés con `JMeter`), usabilidad (heurísticas + accesibilidad) y portabilidad (`cross-browser` + `Docker`).
5. **Seguridad y trazabilidad por diseño.** El control de acceso por JWT y rol, el 2FA para estudiantes, el cifrado de contraseñas, la no exposición de stack traces y la auditoría completa constituyen una base de seguridad robusta y verificada.
6. **Aceptación del usuario.** Los siete criterios de aceptación quedan respaldados por pruebas automatizadas y de sistema, confirmando que SmartComedor satisface las necesidades de los actores del comedor universitario.

**Recomendaciones.** (i) Incrementar la cobertura de ramas en `mealsController` y `auditService`; (ii) añadir pruebas unitarias de componentes React con Testing Library; (iii) ejecutar periódicamente los planes de `JMeter` y `BrowserStack` en integración continua; y (iv) realizar pruebas de usabilidad con usuarios reales (cuestionario SUS) para cuantificar la satisfacción.

## Referencias

- Apache Software Foundation. (2024). *Apache JMeter*. Consultado el 16 de junio de 2026, desde <https://jmeter.apache.org/>
- Beizer, B. (1990). *Software Testing Techniques* (2.<sup>a</sup> ed.). Van Nostrand Reinhold.
- Google. (2024). *Material Design 3*. Consultado el 16 de junio de 2026, desde <https://m3.material.io/>
- IEEE. (2008). *IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008)* (inf. téc.). Institute of Electrical y Electronics Engineers. New York, NY.  
<https://doi.org/10.1109/IEEESTD.2008.4578383>
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models* (inf. téc.). ISO. Geneva, Switzerland.
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308-320. <https://doi.org/10.1109/TSE.1976.233837>
- Meta Open Source. (2024). *Jest: Delightful JavaScript Testing Framework*. Consultado el 16 de junio de 2026, desde <https://jestjs.io/>
- Microsoft. (2024). *Playwright: Fast and Reliable End-to-End Testing for Modern Web Apps*. Consultado el 16 de junio de 2026, desde <https://playwright.dev/>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3.<sup>a</sup> ed.). John Wiley & Sons.
- Nielsen, J. (1994). Heuristic Evaluation. En J. Nielsen & R. L. Mack (Eds.), *Usability Inspection Methods* (pp. 25-62). John Wiley & Sons.
- OWASP Foundation. (2021). *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*. Consultado el 16 de junio de 2026, desde <https://owasp.org/Top10/>
- Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8.<sup>a</sup> ed.). McGraw-Hill Education.
- Sommerville, I. (2011). *Software Engineering* (9.<sup>a</sup> ed.). Addison-Wesley.